

oom-watch

- Why
- Documentation map
- Status
- Quick start
- Build and test
- Layout
- Resources used

oom-watch

A small Go daemon that uses `atop` to detect imminent system overload and write a detailed forensic Markdown report **before** the system actually breaks.

It is the **third leg** of OOM-Protect, alongside `oom-hardening.sh` (the umbrella) and `oom-runner.sh` (the per-leaf cgroup bound).

Why

After the 2026-04-28 user-manager OOM kill (see `../reports/Crash_Report.md`), we wanted *evidence* of what was running in the seconds before the crash — not after, when the kernel had already killed everything. `atop` logs are post-hoc; `oom-watch` writes Markdown reports while the system is still alive but heading off the cliff.

Each report includes a **forensic-detail section** for the top memory consumers — full `/proc/<pid>/cmdline` (the actual argv, not `atop`'s truncated 15-char name), PPID + parent's full cmdline, UID, cgroup path, peak RSS (`VmHWM`), kernel `oom_score` and `oom_score_adj` . Plus a kernel OOM-killer event tail (filtered `dmesg`). The forensic-detail section answers “exactly which binary, with which arguments, started by which script, in which cgroup” — without log archaeology.

Documentation map

Document	Purpose
<code>../manuals/oom-watch-deployment-guide.md</code>	Install, verify, troubleshoot. Read this first. ALT Linux specifics, cross-UID build note, real session walk-through, diagnostic bundler.
<code>../manuals/oom-watch-manual.md</code>	Feature reference. Every config key, severity ladder, report anatomy (including the forensic-detail section), threshold tuning recipes, atop version handling.
<code>../manuals/oom-watch-runbook.md</code>	3 a.m. playbook. 13 incident scenarios with one-command diagnosis, exact remediation, and the regression Challenge that locks each fix in.
<code>../reports/oom-watch-architecture.md</code>	Design rationale: why parser, threshold, sandbox, enrichment, and remediation choices are what they are.
<code>Constitution.md</code> (this dir)	Submodule charter; reiterates anti-bluff Article I.
<code>CLAUDE.md</code> / <code>AGENTS.md</code> (this dir)	Context for AI agents working on the daemon.

All Markdown docs are rendered to HTML and PDF via `make docs`.

Status

- **Tests:** real, anti-bluff (see `Constitution.md` Article I). `go test -count=1 ./...` and `bash ../challenges/run-all.sh` are part of the definition of done. **7/7** Challenges pass; one of them (`challenge-tests-are-not-bluffs.sh`) mutates a parser line and asserts the unit tests turn red — proving the test suite is not a bluff.
- **Dependencies:** Go 1.22+ stdlib only. atop must be installed on the host at runtime.
- **Verified on:** ALT Linux 11 “Salvia” / Sisyphus, kernel 6.12.61, systemd 258, atop 2.12.1, x86_64.
- **Constitution:** the project root `Constitution.md` applies; this submodule has its own `Constitution.md`, `CLAUDE.md`, and `AGENTS.md` reiterating it.

Quick start

The recommended path is the one-shot deployer:

```
sudo make oomwatch-deploy      # or 'make oomwatch-deploy' if already root
```

This runs `oom-watch/scripts/install-and-verify.sh`, which:

1. Pre-flights `atop` / `systemd` / `Go`.
2. Builds `oomwatch` (with `-buildvcs=false` for cross-UID repo resilience).
3. Runs `make oomwatch-install` (idempotent, safe re-run).
4. Validates `/etc/oom-watch/config.json` with `-dry-run` BEFORE asking `systemd` to start the unit. Auto-remediates a broken installed config by backing it up to `config.json.broken.<ts>` and copying the shipped example.
5. `daemon-reload` + `reset-failed` + `enable` + `restart`.
6. Polls `is-active` for up to 30 s; bails fast on `failed`.
7. Asserts the journal contains `"atop located"` (proves the daemon reached the sample loop).
8. Waits up to 60 s for the first report; forces a `-one-shot` if the host is calm.
9. Prints a Summary block.

If anything fails, the EXIT trap dumps `systemctl status`, last 50 journal lines, the config file, the unit file, and a listing of the report directory — all in one place.

For full deployment-failure diagnostics, run:

```
sudo make oomwatch-diagnose
```

This produces `/tmp/oomwatch-diagnose-<ts>.log` containing 16 sections: timestamp, host, tool versions, repo state, shipped example SHA, installed paths, full installed config, dry-run verdict, unit state, full unit file, last 200 journal lines, reports listing, live atop sample, full deploy attempt with all step headers, post-deploy unit state, post-deploy journal.

Build and test

```
make oomwatch-build      # builds oom-watch/oomwatch
make oomwatch-test       # go vet + go test (anti-bluff unit suite)
make challenges           # E2E: 7 / 7 PASS expected
make oomwatch-deploy     # full deploy + verify (sudo)
make oomwatch-diagnose   # bundle current state for support (sudo)
```

Or directly:

```
cd oom-watch
go build -buildvcs=false -o oomwatch ./cmd/oomwatch
go test -count=1 ./...
```

Layout

oom-watch/	
├─ cmd/oomwatch/main.go	entry point + flag parsing
├─ cmd/oommemhog/main.go	in-tree memory hog used by challenge-real-pressure.sh (bounded 16 GiB / 5 min)
├─ internal/atop/	atop -PALL parser + runner (handles atop 2.x per-thread PRM emission via IsLeader filter)
├─ internal/config/	JSON config loader (zero external deps; rejects unknown fields)
├─ internal/monitor/	threshold engine (pure) + main loop with cooldown + escalation
├─ internal/snapshot/	/proc + cgroup + journal capture PLUS forensic enrichment of top-N PIDs (cmdline, status, cgroup, oom_score, parent's cmdline) + kernel OOM dmesg tail
├─ internal/report/	atomic Markdown report writer
├─ internal/logx/	tiny slog wrapper
├─ scripts/install-and-verify.sh	one-shot deployer (make oomwatch-deploy)
├─ scripts/diagnose.sh	full state bundler (make oomwatch-diagnose)
├─ systemd/oom-watch.service	systemd unit (hardened, atop-compatible)
├─ config/oom-watch.example.json	example config (validated by Challenge)
├─ Constitution.md	submodule charter (anti-bluff)
├─ CLAUDE.md	context for Claude Code
├─ AGENTS.md	context for AI agents

Resources used

This daemon was built consulting:

- [atop official repository](#) — parseable output format reference.
- [atop interactive cheatsheet \(PDF\)](#) — for cross-checking key sequences when running atop interactively for triage.
- [atoptool.nl](#) — official site.
- [bytedance/netatop-bpf](#) — eBPF-based per-process network stats (PRN label) — future integration target.
- [pizhenwei/atophttpd](#) — atop served over HTTP — useful pattern if we ever want to expose oom-watch reports remotely.
- [atopsar-plot](#) — retrospective plotting from atop logs; complements oom-watch's incident-time reports.